

Python Companion

Vibe Programming for
the Medical Office

A Practical Guide to the Supplied Programs

George Grätzer

About This Edition

This companion follows the 23 application chapters of *Vibe Programming for the Medical Office*. Chapter numbers match the main book. You do not need to study Python before using the applications.

The printed companion concentrates on running an example, checking its result, and understanding a few important lines. Complete programs, input files, larger practice datasets, and saved reference outputs remain in the accompanying package. Long program listings and raw data tables are not repeated here.

All 25 distinct book programs, their required local copies, the input data, and the optional `run_companion.py` are unchanged from the supplied `Python_Companion(2).zip`. The short edition changes the reading guide, not the applications.

The examples use invented records and supplied teaching rules. They are not medical advice, clinical triage, or authorization for operational use. Do not substitute real patient records for the teaching files.

Original book release: `medical-teaching-2026-09-19-r3`. Short companion: `medical-python-2026-09-19-r2-short`. The original reference interpreter was Python 3.13.5. The final section and `CHECKED_RESULTS.txt` distinguish the new checks from the original release record.

To typeset this guide, keep `Python_Companion.tex` and `Python_Companion.sty` together. The excerpts are embedded: no external program listing, figure folder, or main-book source is required.

Contents

How to Use This Companion

Start with the Result

Open `reference_outputs/ch01/journey_timeline.html` in a browser to see the first example. This is a saved reference result, not a new execution. Then turn to Chapter 1 and run your own copy. Later, use the chapter number from the main book to find the matching guide here.

Each application has a short run-and-check guide followed by selected Python lines and one change to try. Read only the explanations that help with your current question. The excerpts are not complete runnable programs; the full `.py` files are in the named chapter folders.

Keep the Files Together

Extract the ZIP before using it. Copy a whole chapter folder, including its local modules and CSV inputs, rather than copying just the principal program. Each folder's `RUN.txt` gives the exact command.

Folder or file	Purpose
<code>teaching_files</code>	Complete programs and small inputs for Chapters 1–23 and the capstone.
<code>practice_datasets</code>	Four larger, independent examples with their own expected results.
<code>reference_outputs</code>	Saved results for comparison; not a working folder.
<code>run_companion.py</code>	Optional helper that copies and runs a bundled example.

Table 1: The reading guide is short; the complete working files remain available.

A `.py` file holds instructions; a `.csv` file holds data rows; `.html` is a browser page; `.svg` is a drawing; and `.json` holds structured information such as counts or run settings. Do not rename one format as another.

Run with ChatGPT or on Your Computer

For a ChatGPT session with file execution available, attach the whole chapter folder as a ZIP and use a request such as this:

Prompt

In a fresh working folder, run the supplied `journey_view.py` unchanged with `journey_demo.csv`. Return `journey_timeline.html`, `journey_events.csv`, and the actual execution output. Do not substitute a new program. If you cannot execute it, say so rather than presenting a predicted result as a run.

On your computer, use a Python 3 interpreter. The bundled programs need no third-party Python packages. From the extracted companion folder, this optional helper runs Chapter 1 in a new copy:

```
python3 run_companion.py 1 --output first_trial
```

The results are inside `first_trial`. Replace 1 with another chapter number, or use `capstone`, `practice02`, `practice08`, `practice09`, or `practice23`. The target `all` runs the whole bundled set. The output folder must be new and its parent must exist. Use the command for your interpreter when it differs from `python3`.

Without the helper, copy the chapter folder, open a terminal in that copy, and use the command printed in its chapter. Chapter 3 needs additional arguments; the capstone has two commands.

Read, Check, Change

Printed line numbers refer to the original source file. A wrapped line and its continuation arrow are typography, not extra Python instructions. Edit the supplied program file, not a copy reconstructed from the PDF.

The *Check the Result* boxes describe the supplied baseline. Check a named record as well as a total. Keep that run, make one change in a fresh copy, and compare the new result. A refusal to overwrite an earlier output can be the expected result of a test. A successful run is not proof that every rule, input, or interpretation is correct.

Patient-Journey Timeline

The first application turns a small event file into a page that you can open in a browser. A person may have several encounters, and an encounter may have several events. The program draws the dated events and keeps a table underneath them. The table is not an optional decoration: it lets you check what each marker represents.

Start by running the supplied file; you need not understand its browser-page code first.

1.1 Run the Example

Folder: `teaching_files/ch01`.

Inputs: `journey_demo.csv`.

From a fresh copy of this folder, run:

```
python3 journey_view.py
```

Rule to preserve. One input row is one event. The six field names and their order are fixed. Event IDs must be distinct; required values must be present; dates must be exact calendar dates in YYYY-MM-DD form. This first program expects a clean file. It does not perform the encounter-anchor checks introduced in Chapter 2. Same-day positions do not imply a time-of-day order.

1.2 Check the Result

Outputs: `journey_events.csv`, `journey_timeline.html`.

Check the meaning

The CSV must contain all 18 events: three people and six encounters. Find E02 and check P01, encounter C01, June 2, 2026, and the detail “Sample collected.” Selecting P01 in the page should show six records. A marker can represent more than one event of the same date and type; do not use the number of circles as the record count.

1.3 Read the Key Lines

```
12 def read_events(path):
13     with Path(path).open(newline="", encoding="utf-8-sig") as stream:
14         reader = csv.DictReader(stream, strict=True)
15         if reader.fieldnames != FIELDS:
16             raise ValueError("Expected header: " + ",".join(FIELDS))
17         rows = list(reader)
18     if any(None in row or None in row.values() for row in rows):
19         raise ValueError("Malformed CSV row")
20     return rows
```

Excerpt from journey_view.py, source lines 12–20.

Named fields rather than column numbers. `csv.DictReader` makes each row a dictionary. Thus `row["event_id"]` retrieves a named field rather than “column 1.” `FIELDS` is the declared header; comparing the received header with it makes a mistaken input fail visibly.

Lists, sets, and groups. The list `rows` retains every record. A set of person references counts distinct people, whereas `len(rows)` counts events. `defaultdict(list)` collects events sharing a date and type. These three operations answer different questions; none licenses deleting a source event.

A page is also an output file. `timeline_svg` constructs vector drawing instructions. `render_page` combines them with tables and a small JavaScript filter. Python creates the page once; JavaScript changes what the browser displays afterward. `escape` ensures that supplied detail text is displayed as text rather than interpreted as markup.

1.4 Try One Change

In a fresh copy, change only E02’s detail to “Sample collected: teaching revision.” Run again. The new text must appear in both the CSV and browser page, while the totals remain three people, six encounters, and 18 events. This separates a presentation change from a change in the population being displayed.

Timeline Checker

The second program deals with records that cannot safely be placed on a timeline. It does not make troublesome rows disappear. Instead, it creates a checking report with the original values, source-row numbers, dispositions, and reasons, and passes that report to the display functions from Chapter 1.

2.1 Run the Example

Folder: `teaching_files/ch02`.

Inputs: `journey_problem.csv`.

Local modules: `journey_view.py`.

From a fresh copy of this folder, run:

```
python3 journey_check.py
```

Rule to preserve. Every input row must survive in the report. Each encounter needs exactly one visit row that itself passes the checks. Other events must agree with that visit’s person and must not precede its date. A duplicated event ID affects both occurrences. An event whose person disagrees with its visit is grouped under the visit owner, while its supplied person remains visible.

2.2 Check the Result

Outputs: `journey_checked.html`, `journey_review.csv`.

Check the meaning

The baseline retains 18 rows: 12 placed and six for review. Review source rows 2, 3, 8, 11, 14, and 18. The two occurrences of E02 are rows 2 and 14; neither is silently selected as the “real” one. In row 18, E18 retains supplied person P01 but has owner P03 and the reason “person differs from visit.”

2.3 Read the Key Lines

```

8 def check_events(rows):
9     ids = Counter(r["event_id"].strip() for r in rows)
10    issues = [[] for _ in rows]
11    days = []
12    visits = defaultdict(list)
13    for i, row in enumerate(rows):
14        for field in FIELDS:
15            if not row[field].strip():
16                issues[i].append("missing " + field)
17        for field in ["event_id", "person_ref", "encounter_id"]:
18            if row[field] != row[field].strip():
19                issues[i].append("padded " + field)
20        key = row["event_id"].strip()
21        if key and ids[key] > 1:
22            issues[i].append("duplicate event_id")

```

Excerpt from journey_check.py, source lines 8–22.

Count before deciding. `Counter` counts all supplied IDs before any row is classified. This avoids a common asymmetric mistake: accepting the first duplicate and rejecting only the second. The parallel `issues` lists allow several reasons to be retained for one row.

Separate passes have different jobs. The first pass checks fields and collects visit positions. The anchor-building pass admits only an unambiguous, otherwise valid visit. The final pass compares each event with that anchor. Reading the passes in that order is more useful than treating the function as one long loop.

Derived and supplied fields can coexist. `dict(row, ...)` preserves the original dictionary fields and adds checking fields. `owner_ref` serves the display; it does not overwrite `person_ref`. That distinction is what makes the mismatch auditable.

2.4 Try One Change

Change only the event ID in source row 14 from E02 to E14. Both duplicate-ID reasons should clear, leaving 14 placed rows and four review rows. The row count stays 18. A result of 13 placed rows would suggest that only one occurrence was reconsidered.

Reusable Journey Tool

The reusable journey tool accepts an input filename, a new output-folder name, and an optional person reference. It saves the report and page together with a record of the input, program versions, interpreter, rules, and counts.

3.1 Run the Example

Folder: `teaching_files/ch03`.

Inputs: `journey_problem.csv`.

Local modules: `journey_check.py`, `journey_view.py`.

From a fresh copy of this folder, run:

```
python3 journey_report.py journey_problem.csv first_run
```

Rule to preserve. Validate the complete input before selecting a person. Selection uses the derived visit owner, not an unverified supplied person. The destination folder must be new, and its parent must already exist. A run record identifies the bytes processed; it does not prove that the rules are appropriate.

3.2 Check the Result

Outputs: `first_run/journey_report.csv`, `first_run/journey_timeline.html`, `first_run/run_record.json`.

Check the meaning

The all-person run retains 18 selected records, 12 placed and six for review. In `run_record.json`, check the input filename, the three program hashes, the stated rule, and the counts. A P03 run must report 18 input records but only six selected records, of which four are placed and two require review. E18 must remain in that selection with its supplied P01 value visible.

3.3 Read the Key Lines

```

11 def build_report(source, destination, person=None):
12     source, destination = Path(source), Path(destination)
13     report = check_events(read_events(source))
14     # Validate the complete file BEFORE selecting a person's records.
15     selected = [r for r in report if person is None or r["owner_ref"]==person]
16     if person is not None and not selected:
17         raise ValueError("No records for selected person: " + person)
18     counts = dict(input_records=len(report), selected_records=len(selected),
19                   placed=sum(r["status"]=="placed" for r in selected),
20                   review=sum(r["status"]=="review" for r in selected))

```

Excerpt from journey_report.py, source lines 11–20.

Arguments are inputs to a function. `build_report(source, destination, person=None)` names three inputs. The default `None` means no person filter. It does not mean that missing person values in the data are acceptable.

Filtering comes after validation. The call to `check_events` precedes the list comprehension that selects records. Reversing those operations could hide a duplicate or an inconsistent anchor outside the chosen person's apparent rows.

Exclusive folder creation. `destination.mkdir(parents=False, exist_ok=False)` refuses a reused output folder. The function then writes all three result files there. If a run fails after folder creation, preserve that partial folder and use a new name for the next attempt.

3.4 Try One Change

Run `python3journey_report.pyjourney_problem.csvP03_run--personP03` in a fresh chapter copy. Check the P03 counts above. Then repeat the identical command: it must fail rather than overwrite `P03_run`. Verify that the earlier files are unchanged. For another successful run, choose a different destination name.

Appointment Report

This report illustrates a subtle programming error: making a percentage look better by dropping an eligible record whose outcome is unresolved. The program records eligibility, confirmed completion, and unresolved information as separate fields. They are not mutually exclusive categories.

4.1 Run the Example

Folder: `teaching_files/ch04`.

Inputs: `appointments.csv`.

From a fresh copy of this folder, run:

```
python3 appointment_report.py
```

Rule to preserve. Eligibility requires an acceptable appointment ID, scheduled date, and status. A missing completion date does not by itself remove an otherwise eligible completed appointment from the denominator. A confirmed completion additionally needs a valid completion date that does not precede the appointment. Both occurrences of a duplicated appointment ID are unresolved and ineligible under this teaching rule.

4.2 Check the Result

Outputs: `appointment_report.csv`.

Check the meaning

The baseline is 2 confirmed completions among 5 eligible records, or 40.0%. A03 has `eligible=1`, `completed=0`, and `unresolved=1`. A04 and A05 remain eligible despite cancelled and no-show statuses. Both A07 rows remain in the output. Four unresolved rows are not the complement of the five eligible rows.

4.3 Read the Key Lines

```
52     eligible = not issues
53     completed = False
54     if status == "completed":
55         try:
56             finished = parse_day(row["completed_date"])
57             if eligible and finished < scheduled:
58                 issues.append("completion precedes appointment")
59             elif eligible:
60                 completed = True
61         except ValueError:
62             issues.append("missing or invalid completion date")
63     output.append(dict(row, data_row=number,
64                       eligible=int(eligible), completed=int(completed),
65                       unresolved=int(bool(issues)), reason="; ".join(issues)))
```

Excerpt from appointment_report.py, source lines 52–65.

Order of assignment matters. The assignment `eligible=notissues` occurs before completion-date checks. This is intentional: those later checks affect confirmation and unresolved status without retroactively changing the defined denominator.

Boolean values support counting. The program stores `int(eligible)` and `int(completed)` as 0 or 1. Summing these fields counts rows meeting the respective rules. It is still necessary to inspect which rows supplied the ones.

An undefined proportion needs a branch. The final expression calculates a proportion only when the denominator is nonzero. The other branch prints “not calculable.” An empty eligible population is not a 0% completion rate.

4.4 Try One Change

In the synthetic A03 row, supply completion date 2026-09-01. The confirmed numerator becomes 3 while the eligible denominator remains 5: 60.0%. Seven or eight eligible rows would indicate that unrelated ID or status rules changed as well.

Measurement Catalogue

A measurement catalogue tells you what is present before you attempt calculations. Here an observation, a person, and an encounter are three different units. The program groups observations by the pair consisting of the measure name and its supplied unit, and creates a second file linking every observation to its catalogue group.

5.1 Run the Example

Folder: `teaching_files/ch05`.

Inputs: `measurements.csv`.

From a fresh copy of this folder, run:

```
python3 measurement_catalogue.py
```

Rule to preserve. Observation IDs must be distinct and required labels must be nonblank and unpadded. Repeated person or encounter references are allowed. Temperature in C and temperature in F are separate catalogue groups. A blank value counts as missing, not as a numerical zero.

5.2 Check the Result

Outputs: `measurement_catalogue.csv`, `observation_lineage.csv`.

Check the meaning

Six observations represent three people and four encounters. There are three measure-unit groups. The pulse group has three supplied observations. Temperature in C has two observations, one supplied and one missing; temperature in F has one supplied observation. The lineage file must still have all six observation IDs.

5.3 Read the Key Lines

```
42 groups = defaultdict(list)
43 for row in rows:
44     groups[(row["measure"], row["unit"])].append(row)
45 catalogue = []
46 for (measure, unit), group in sorted(groups.items()):
47     catalogue.append(dict(measure=measure, unit=unit,
48         observations=len(group),
49         supplied=sum(bool(r["value"].strip()) for r in group),
50         missing=sum(not r["value"].strip() for r in group),
51         sources="; ".join(sorted({r["source"] for r in group}))))
```

Excerpt from measurement_catalogue.py, source lines 42–51.

A tuple is a grouping key. `(row["measure"], row["unit"])` is one key with two components. Grouping on the measure name alone would merge incompatible supplied units before any conversion rule had been stated.

Distinct counts differ from row counts. A set of person references has three elements, while the list of observations has six. `require_ids` is applied to observation IDs, not to person references. Applying it to the latter would reject legitimate repeated observations.

Presence is a modest claim. `bool(r["value"].strip())` tests whether text is supplied. It does not show that the text is numeric or meaningful. The separate measurement checker in Chapter 6 illustrates a stronger claim.

5.4 Try One Change

Copy the first observation as a new synthetic observation O07, changing its value to 72 but keeping the same person, encounter, measure, unit, and source. Expect seven observations, still three people and four encounters, still three groups, and four observations in the pulse group. The lineage file must gain O07.

Measurement Checks

This checker records several kinds of data problem without stopping at the first problematic row. It distinguishes malformed dates, missing values, nonnumeric values, non-finite numbers, unrecognized units, and ID problems. The output says “passes checks,” deliberately not “clinically valid.”

6.1 Run the Example

Folder: `teaching_files/ch06`.

Inputs: `measurements_to_check.csv`.

From a fresh copy of this folder, run:

```
python3 measurement_check.py
```

Rule to preserve. Each of the eight source rows must remain visible. A numeric value must be finite; a date must be an exact valid calendar date; units must be one of the supplied labels C and F. Both duplicate-ID occurrences are flagged. No physiological range, conversion, or treatment rule is implemented.

6.2 Check the Result

Outputs: `measurement_check.csv`.

Check the meaning

M01 and M02 pass the stated checks. The remaining six rows require review: M03 has an impossible date, M04 a missing value, M05 an unrecognized unit, M06 nonnumeric text, and both M07 occurrences a duplicate ID. Count rows with reasons, not the number of semicolon-separated reasons.

6.3 Read the Key Lines

```
50     if not row["value"].strip():
51         reasons.append("missing value")
52     else:
53         try:
54             if not math.isfinite(float(row["value"])):
55                 reasons.append("non-finite value")
56         except ValueError:
57             reasons.append("not numeric")
58     if row["unit"] not in {"C", "F"}:
59         reasons.append("unrecognized unit")
```

Excerpt from measurement_check.py, source lines 50–59.

Conversion can fail. `float` and date parsing can raise `ValueError`. The `try/except` blocks turn these particular failures into row-level reasons, allowing the checker to continue with later rows.

Numeric does not imply finite. Values such as a non-finite floating-point token may convert successfully. The extra `math.isfinite` check makes the numeric contract explicit. This is a different check from testing whether text was supplied.

Schema failure differs from a review row. A wrong header or malformed CSV row raises an exception in `read_rows`. The program can retain a row-level value problem only after it knows which fields that row actually contains.

6.4 Try One Change

For this invented fixture only, stipulate that M05's supplied unit label K was a transcription error and replace it with C. Expect three rows passing the checks and five review rows. This is a stated source correction, not an automatic conversion between units; the program performs no such conversion.

Weekly Teaching Extract

The weekly teaching extract demonstrates selecting a period and exporting only the fields needed for a particular count. The synthetic input contains names, email addresses, and free text. None belongs in the requested output.

7.1 Run the Example

Folder: `teaching_files/ch07`.

Inputs: `synthetic_schedule.csv`.

From a fresh copy of this folder, run:

```
python3 minimal_extract.py
```

Rule to preserve. Include events dated in September 2026, and group them by Monday week start and supplied status. Allowed statuses are completed, cancelled, and no_show. Event IDs must be distinct. The exported columns are exactly `week_start`, `status`, and `count`; names, contacts, and free text are excluded.

7.2 Check the Result

Outputs: `extract_receipt.csv`, `weekly_counts.csv`.

Check the meaning

Five of six source events are in September; one is outside the period. The first grouped week is labelled 2026-08-31 because that Monday begins the week containing early September dates. Its two counts are one completed and one no-show. The week beginning September 7 has two completed and one cancelled event.

7.3 Read the Key Lines

```
45 counts = Counter()
46 included = 0
47 for row in rows:
48     day = parse_day(row["event_date"])
49     if row["status"] not in allowed:
50         raise ValueError("Unexpected status")
51     if day.year == 2026 and day.month == 9:
52         monday = day - timedelta(days=day.weekday())
53         counts[(monday.isoformat(), row["status"])] += 1
54         included += 1
55 report = [dict(week_start=week, status=status, count=count)
56           for (week, status), count in sorted(counts.items())]
```

Excerpt from minimal_extract.py, source lines 45–56.

Filter by event date, label by week. The month test uses the event’s date. Only afterward does `day-timedelta(days=day.weekday())` compute the Monday label. Filtering on the week label instead would incorrectly lose September events from the August 31 week.

Count a pair of categories. `Counter` uses the pair of week and status as its key. Each included row increments exactly one cell. The sorted keys produce a stable output order independent of source order.

Construct an allowed output. The output dictionaries contain only the three requested fields. This is different from copying each source row and attempting to remove sensitive-looking columns afterward.

7.4 Try One Change

Reverse the order of the six input records and run in another fresh copy. The grouped-count CSV should be unchanged byte for byte. Input order is not part of this report’s grouping rule, although a future source-row ledger would need to record the new row positions.

Follow-Up Matching

This application links candidate follow-ups to encounters. The relationship is defined by encounter ID, not by a shared person reference or a guess based on proximity. The program retains a summary for every encounter, a ledger of candidates with known encounters, and a separate file for unknown references.

8.1 Run the Example

Folder: `teaching_files/ch08`.

Inputs: `encounters.csv`, `followups.csv`.

From a fresh copy of this folder, run:

```
python3 followup_match.py
```

Rule to preserve. With the default window, elapsed days 0 through 14 are inclusive. Among eligible candidates for one encounter, choose the smallest pair of elapsed days and follow-up ID. Both input files require distinct record IDs. An unknown encounter reference goes to the separate unmatched file, not into a newly invented encounter.

8.2 Check the Result

Outputs: `followup_candidates.csv`, `followup_matches.csv`, `unmatched_followups.csv`.

Check the meaning

E01 selects F01 at day 0; E02 selects F02 at day 14; E05 selects F05 at day 3 rather than F06 at day 5. E03's F03 is at day 15 and E04's F04 is at day -1, so neither is selected. E06 has no candidate. F07 refers to unknown E99 and must appear in the unmatched file.

8.3 Read the Key Lines

```

65     for event in followups:
66         if event["encounter_id"] != encounter["encounter_id"]:
67             continue
68         days = (parse_day(event["followup_date"]) - start).days
69         eligible = 0 <= days <= args.window_days
70         candidates.append(dict(encounter_id=encounter["encounter_id"],
71                               followup_id=event["followup_id"], elapsed_days=days,
72                               within_window=int(eligible)))
73         if eligible:
74             valid.append((days, event["followup_id"]))
75     valid.sort()
76     matches.append(dict(encounter_id=encounter["encounter_id"],
77                       candidates_in_window=len(valid),
78                       selected=valid[0][1] if valid else "",
79                       elapsed_days=valid[0][0] if valid else "",
80                       status="matched" if valid else "no match in window"))

```

Excerpt from followup_match.py, source lines 65–80.

Boundary comparisons carry the rule. `0<=days<=args.window_days` includes both endpoints. The date library supplies a difference in days; the comparison supplies the inclusion decision.

Tuple sorting supplies a tie-break. Each valid candidate is stored as `(days, followup_id)`. Sorting these pairs gives an earliest-day choice and then an ID tie-break. Source order does not become an accidental priority rule.

Not selected is not the same as unmatched. A known candidate can be eligible but not selected, outside the window, or before the index date. The final unmatched file has the narrower meaning of an unknown encounter reference. These categories should not be collapsed.

8.4 Try One Change

Run `python3followup_match.py--window-days15` in a fresh copy. E03 should now select F03 and the matched encounter count should rise from three to four. E04 remains before the index date; F07 remains an unknown reference. The changed result belongs to a different, explicitly stated teaching contract.

Cohort Summary

The cohort summary separates membership in a population from availability of a particular measurement. The input contains one row per person. The output includes a disposition ledger, a count of dispositions, and summary measures with their respective population sizes.

9.1 Run the Example

Folder: `teaching_files/ch09`.

Inputs: `cohort_candidates.csv`.

From a fresh copy of this folder, run:

```
python3 cohort_summary.py
```

Rule to preserve. A person is included when age is supplied and at least 18, and enrollment is yes. Missing age is unresolved rather than assumed adult. Exclusion reasons are assigned in the order age unresolved, under 18, not enrolled, included. Person references must be unique. Observed waits must be finite and nonnegative.

9.2 Check the Result

Outputs: `cohort_flow.csv`, `cohort_ledger.csv`, `cohort_summary.csv`.

Check the meaning

Twelve candidates yield eight eligible people. Six of the eight have recorded follow-up, and six have observed waits. The median of those six waits is 9 days. S04 and S09 are eligible despite missing waits; S07 is not silently included with an invented age. The ledger must contain all twelve person references.

9.3 Read the Key Lines

```

64 waits = [float(r["wait_days"]) for r in eligible if r["wait_days"].strip()]
65 followed = sum(r["followup_recorded"] == "yes" for r in eligible)
66 summary = [
67     dict(measure="candidates", value=len(rows), basis_n=len(rows)),
68     dict(measure="eligible", value=len(eligible), basis_n=len(rows)),
69     dict(measure="followup_recorded", value=followed, basis_n=len(eligible)),
70     dict(measure="observed_waits", value=len(waits), basis_n=len(eligible)),
71     dict(measure="median_wait_days", value=statistics.median(waits) if waits
    ↪ else "",
72         basis_n=len(waits))]

```

Excerpt from cohort_summary.py, source lines 64–72.

Two selections answer different questions. The eligible list is formed from membership rules. The waits list then selects observed measurements within that population. Computing the latter directly from all source rows would include waits from excluded candidates.

A median has a basis, not a proportion denominator. `statistics.median(waits)` uses the observed values. The `basis_n` field reports how many values support it. The recorded-follow-up count has the eligible cohort as its basis instead.

Missing information is not zero. The blank wait test prevents a missing value from becoming zero. Zero would be a genuine observed wait and would affect the median. A blank age likewise prevents the membership decision from being made.

9.4 Try One Change

Stipulate an age of 18 for synthetic S07 and change that one field. Expect nine eligible people, seven recorded follow-ups, and seven observed waits. Their median becomes 8 days. This change affects both cohort membership and the observed-value list, so retaining the old median would be a warning sign.

Monthly Waiting-Time Report

The waiting-time report builds a complete twelve-month view even though one month has no supplied observations. It writes a numeric table and an SVG chart from the same summary. The missing month remains a gap, not a zero and not a line interpolated between neighboring months.

10.1 Run the Example

Folder: `teaching_files/ch10`.

Inputs: `waiting_times.csv`.

From a fresh copy of this folder, run:

```
python3 waiting_time_report.py
```

Rule to preserve. The report covers the twelve months of 2026. Month labels must be exact YYYY-MM strings; waits must be finite and nonnegative. Each month gets an output row containing its count and, only when values exist, its median. Missing months break the plotted line.

10.2 Check the Result

Outputs: `monthly_waits.csv`, `waiting_time_report.svg`.

Check the meaning

There are eleven observed months. July has count 0 and an empty median, while June's median is 11 and August's is 10. The chart must not draw a point at zero for July or connect June directly to August. January has two observations and a median of 6.

10.3 Read the Key Lines

```
43 summary = []
44 for m in range(1,13):
45     key = f"2026-{m:02}"
46     values = groups[key]
47     summary.append(dict(month=key, count=len(values),
48                         median_days=statistics.median(values) if values else ""))
49 save_rows("monthly_waits.csv", ["month","count","median_days"], summary)
```

Excerpt from waiting_time_report.py, source lines 43–49.

Enumerate the reporting frame explicitly. The loop over `range(1,13)` creates every month, not just keys that happened to appear in the input. This is how the program makes an absent month visible.

Choose the empty-group representation. An empty values list gives count 0 and an empty median field. The code does not call the median function on that empty list or supply a made-up result.

The chart state has a meaning. `previous` stores the previous plotted point. When the month has no observations it is reset to `None`, preventing a segment from bridging the gap.

10.4 Try One Change

Add one invented July observation with wait 0. July should now have count 1 and median 0, and the chart should have an actual July point. This is deliberately different from the baseline's missing July. The exercise tests representation of supplied zero, not a proposal to fill missing real records with zero.

Weekly Follow-Up Monitor

The weekly monitor starts with supplied aggregate counts. It preserves provisional weeks but excludes them from the final pooled result. Its main calculation is a ratio of totals, not the unweighted average of weekly percentages.

11.1 Run the Example

Folder: `teaching_files/ch11`.

Inputs: `weekly_followup.csv`.

From a fresh copy of this folder, run:

```
python3 weekly_followup.py
```

Rule to preserve. Week numbers must be distinct positive integers. Counts must be nonnegative integers, and completions cannot exceed the eligible count. A zero eligible count produces an empty weekly rate. Only rows with `provisional=no` contribute to the final pooled numerator and denominator.

11.2 Check the Result

Outputs: `weekly_followup_report.csv`, `weekly_summary.csv`.

Check the meaning

The ten final weeks contribute 105 completions among 130 eligible records: 80.8% at one decimal place. Weeks 11 and 12 remain in the detailed report as provisional. The unresolved counts and the provisional flags are supplied information, not values inferred from today's date.

11.3 Read the Key Lines

```

45 final=[r for r in report if r["provisional"]=="no"]
46 num=sum(r["completed"] for r in final); den=sum(r["eligible"] for r in final)
47 summary=[dict(measure="final_weeks",value=len(final)),
48           dict(measure="final_completed",value=num),
49           dict(measure="final_eligible",value=den),
50           dict(measure="pooled_final_percent",value=round(100*num/den,1) if den
    ↪ else ""),
51           dict(measure="provisional_weeks",value=sum(r["provisional"]=="yes" for
    ↪ r in report))]

```

Excerpt from weekly_followup.py, source lines 45–51.

Keep the detailed report before pooling. The full report is saved first. The `final` list is a second view selected from it. Excluding provisional weeks from a final summary therefore does not remove them from the handover.

Pool counts, not rounded percentages. The program sums `completed` and `eligible` and divides once. Averaging `rate_percent` would give each week equal weight and would also use already-rounded values.

Do not infer meanings absent from the input. The program checks that unresolved counts are nonnegative, but does not derive them or equate them with `eligible` minus `completed`. The source contract must say what they count.

11.4 Try One Change

Change only week 11's provisional flag to no. The final totals become 113 completions among 142 eligible records, or 79.6%; one provisional week remains. Do not change the counts merely to keep the original percentage.

Clinic Schedules

The schedule comparison uses one invented arrival file to run two non-preemptive service rules. One chooses the earliest waiting arrival; the other gives precedence to already-waiting records bearing the supplied urgent label. The program does not assign that label or implement clinical triage.

12.1 Run the Example

Folder: `teaching_files/ch12`.

Inputs: `clinic_arrivals.csv`.

From a fresh copy of this folder, run:

```
python3 schedule_comparison.py
```

Rule to preserve. There is one server, positive integer service duration, and no interruption of a visit already in service. Only arrived visits are eligible to be chosen. Ties are resolved by arrival time and visit ID. A server with nobody waiting advances to the next arrival time.

12.2 Check the Result

Outputs: `schedule_detail.csv`, `schedule_summary.csv`.

Check the meaning

First-come order is A, B, C, D, E, F. The alternative order is A, C, E, B, D, F. Mean waits are 18 and 15.5 minutes, but maximum waits are 25 and 28. D waits 23 minutes under the first rule and 28 under the second. Those figures describe this invented scenario, not a generally preferable policy.

12.3 Read the Key Lines

```

47     ready=[r for r in pending if r["arrival_minute"]<=clock]
48     if not ready:
49         clock=min(r["arrival_minute"] for r in pending)
50         continue
51     if rule=="urgent_first":
52         key=lambda r:(r["priority"]!="urgent",r["arrival_minute"],r["
↪ visit_id"])
53     else:
54         key=lambda r:(r["arrival_minute"],r["visit_id"])
55     chosen=min(ready,key=key)

```

Excerpt from schedule_comparison.py, source lines 47–55.

Simulation state is explicit. pending holds unserved visits and clock records the next available service time. A visit is removed exactly once after its finish time is calculated.

Do not choose a future arrival. The ready list is formed before the selection key is applied. Sorting the entire original file by priority would let a future arrival jump ahead of someone who could already be served.

Boolean keys can express precedence. For the alternative rule, `priority!="urgent"` sorts supplied urgent records before routine ones. Arrival time and ID complete the key and make ties deterministic.

12.4 Try One Change

Set all six supplied priority labels to routine in a fresh copy. The two rules should now produce the same order and the same overall mean of 18 minutes. Both urgent-group means should be empty. This checks that the difference between the rules really comes from the supplied labels.

Appointment Letters

This program creates a local HTML pack of appointment letters from invented requests. It checks the required details, holds incomplete or invalid requests, and records every request in a build report. It does not connect to an email service, print automatically, or send messages.

13.1 Run the Example

Folder: `teaching_files/ch13`.

Inputs: `letter_requests.csv`.

From a fresh copy of this folder, run:

```
python3 appointment_letters.py
```

Rule to preserve. Appointment IDs are distinct. A letter requires a name, exact date and time, location, and contact. Incomplete requests are held with reasons rather than completed by invented details. Supplied text is escaped before being inserted into HTML.

13.2 Check the Result

Outputs: `appointment_letters.html`, `letter_build_report.csv`.

Check the meaning

L01, L02, and L03 generate letters. L04 is held because its contact field is blank. The build report contains all four request IDs. Check L01's October 1, 2026, 09:00 appointment and supplied location against the input; do not infer that the generated letter has been sent.

13.3 Read the Key Lines

```

49     status="hold" if issues else "generated"
50     report.append(dict(appointment_id=row["appointment_id"],status=status,
51                       reason="; ".join(issues)))
52     if issues:
53         continue
54     fields={k:escape(v) for k,v in row.items()}
55     sections.append("<section><p>SYNTHETIC TEACHING EXAMPLE</p><h1>Appointment
    ↳ details</h1>"+
56         "<p>"+fields["display_name"]+"</p><p>"+fields["date"]+" at "+fields["
    ↳ time"]+
57         "</p><p>Location: "+fields["location"]+"</p><p>Questions or changes: "+
58         fields["contact"]+"</p><p>Please check these details and contact the
    ↳ office "+
59         "if anything is incorrect.</p><p>Reference: "+fields["appointment_id"]+
    ↳ "</p></section>")

```

Excerpt from appointment_letters.py, source lines 49–59.

A list comprehension can collect missing fields. The `issues` list initially contains the names of blank required fields. Date-time parsing can add another reason. A nonempty list controls both the hold status and the decision not to generate a section.

Escape data, not the template. The fixed HTML template contains markup. The supplied fields are passed through `escape` before insertion so that names or locations containing angle brackets are not interpreted as instructions.

One report row for each request. The build-report row is appended before `continue` skips a held letter. Reversing that order could make held requests vanish from the report.

13.4 Try One Change

Supply the invented contact 555-0101 for L04. Four letters should now be generated and no request held. Check the added letter's details. The change must not add message-sending behavior.

Rule-and-Boundary Lab

The rule-and-boundary lab implements a deliberately fictional classroom score. Its purpose is to make boundary conditions and component calculations visible. It does not represent a validated clinical score and should not be renamed to resemble one.

14.1 Run the Example

Folder: `teaching_files/ch14`.

Inputs: `score_cases.csv`.

From a fresh copy of this folder, run:

```
python3 teaching_score.py
```

Rule to preserve. Duration below 5 gives 0 points, duration from 5 up to but not including 10 gives 1, and duration at least 10 gives 2. Task counts through 2 give 0, through 5 give 1, and higher counts give 2. Supplied support=no adds 1. Invalid input prevents calculation.

14.2 Check the Result

Outputs: `teaching_score_report.csv`.

Check the meaning

C01 totals 0, C02 totals 1, C03 totals 5, and C04 totals 2. C05 has missing duration; C06 has negative duration. Notice the exact boundaries: C02's duration is 5 and C03's is 10. A program using the wrong comparison could look correct on most other inputs.

14.3 Read the Key Lines

```
56     d = t = support = total = ""
57     if not issues:
58         d = 0 if duration < 5 else 1 if duration < 10 else 2
59         t = 0 if tasks <= 2 else 1 if tasks <= 5 else 2
60         support = int(row["support_present"] == "no")
61         total = d+t+support
62     report.append(dict(case_id=row["case_id"],duration_points=d,
63                       task_points=t,support_points=support,total=total,
64                       status="review" if issues else "calculated",
65                       reason="; ".join(issues)))
```

Excerpt from teaching_score.py, source lines 56–65.

Validate before assigning components. The component variables start as empty strings. They receive numeric values only when `issues` is empty. This prevents a missing duration from accidentally behaving like duration 0.

Nested conditional expressions encode intervals. The duration expression tests the lower interval first, then the middle interval, then the remaining upper interval. Read it as three disjoint cases rather than as a compact formula to admire.

Components make a total inspectable. Saving the separate points lets you locate an incorrect comparison without reverse-engineering the sum. A matching total alone can hide two compensating errors.

14.4 Try One Change

Change C02's duration from 5 to 4.9. Its duration component and total should fall from 1 to 0; every other case should remain unchanged. This is a test of the stated interval boundary, not a new scoring recommendation.

Predictive-Value Explorer

The predictive-value explorer is a fictional expected-count calculation, not an evaluation of a real diagnostic test. Python writes a scenario CSV and a browser page. The browser then lets a reader try other prevalence values under the same supplied model.

15.1 Run the Example

Folder: `teaching_files/ch15`.

Inputs: `test_scenarios.csv`.

From a fresh copy of this folder, run:

```
python3 predictive_value_explorer.py
```

Rule to preserve. The baseline assumes 10,000 people, sensitivity 0.90, and specificity 0.95. Prevalence must be between 0 and 100 percent. Expected cells may be fractional. The positive predictive value is calculated from expected true positives divided by all expected positives, and is left undefined when that denominator is zero.

15.2 Check the Result

Outputs: `predictive_value.html`, `test_scenarios_report.csv`.

Check the meaning

At 1% prevalence, expected true positives are 90 and expected false positives 495, giving 15.4%. The remaining cells are 10 and 9,405, and all four sum to 10,000. At 10% and 30% prevalence the saved percentages are 66.7% and 88.5%. Changing the browser control does not rewrite the separately saved scenario CSV.

Inspect both the saved CSV and the interactive page; a correct CSV does not test the browser controls.

15.3 Read the Key Lines

```
31 def expected(prevalence):  
32     condition = MODEL["n"] * prevalence  
33     tp = condition * MODEL["sensitivity"]  
34     fn = condition - tp  
35     fp = (MODEL["n"] - condition) * (1 - MODEL["specificity"])  
36     tn = MODEL["n"] - condition - fp  
37     return tp, fp, fn, tn
```

Excerpt from predictive_value_explorer.py, source lines 31–37.

One model, two execution environments. The Python `expected` function computes the saved scenarios. `json.dumps(MODEL)` embeds the same assumptions in the page’s JavaScript. The two calculations still need comparison; sharing assumptions does not itself prove identical behavior.

Expected counts precede rounding. The code calculates the four cells from unrounded quantities and forms the ratio before rounding its displayed percentage. Rounding cells too early can change the reported proportion.

Undefined is not zero. The branch on the positive total protects the ratio when no positives are expected. A blank CSV value and the page’s “undefined” label express absence of a defined ratio, not a 0% estimate.

15.4 Try One Change

Add scenario rows with prevalence 0 and 100. Under the unchanged baseline assumptions, the corresponding saved positive predictive values are 0% and 100%. Check all four expected cells and their total as well. Compare the same values in the browser. Changing its control does not rewrite the saved CSV.

Literature-Review Organizer

The literature-review organizer groups invented retrieved records by a supplied study ID. Multiple records can concern one study. The program preserves those record identities and highlights disagreements instead of silently keeping the first record and discarding the others.

16.1 Run the Example

Folder: `teaching_files/ch16`.

Inputs: `citations.csv`.

From a fresh copy of this folder, run:

```
python3 review_organizer.py
```

Rule to preserve. Record IDs are distinct and study IDs are present. Decisions must be include, exclude, or pending; exclusions require reasons. A disagreement in decision, title, or year within a study group produces review conflict, together with the differing values and their source record IDs.

16.2 Check the Result

Outputs: `study_screening.csv`.

Check the meaning

Eight retrieved records form six study groups. ST01 contains R01 and R02; ST03 contains R04 and R06. The baseline has three included, two excluded, one pending, and no conflicting groups. The two additional retrieved records remain listed within their study groups rather than being erased.

16.3 Read the Key Lines

```
49     conflict_fields = [field for field in ["decision", "title", "year"]
50                        if len({r[field] for r in group}) > 1]
51     conflict = bool(conflict_fields)
52     differences = "; ".join(field + ": " + " , ".join(
53         r["record_id"] + "=" + r[field] for r in group)
54         for field in conflict_fields)
55     decision = "review conflict" if conflict else group[0]["decision"]
```

Excerpt from review_organizer.py, source lines 49–55.

Group first, inspect differences second. `defaultdict(list)` retains all records for each study. Sets of supplied values then reveal whether a field agrees within that group. This is not the same as deduplicating whole rows.

A conflict has named fields. `conflict_fields` records which of decision, title, and year differ. The program then constructs strings showing each record ID and supplied value, making the flag inspectable.

Sorting can stabilize a report. Sorting study IDs and sets of reasons gives predictable output order. It does not decide which conflicting value is correct.

16.4 Try One Change

Change only R02's year from 2024 to 2023. ST01 should become review conflict with year named as the conflicting field; both R01 and R02 must remain. Expect two included groups, two excluded, one pending, and one conflict. Do not resolve the conflict by deleting R02.

Reproducible Research Table

The research table aligns baseline and follow-up measurements by person reference and computes a change only for complete pairs. It saves a person-level table, a small summary, and a run record identifying the source and program.

The calculation is descriptive and uses invented teaching units. A mean change in these records is not evidence of a treatment effect or a substitute for a research design.

17.1 Run the Example

Folder: `teaching_files/ch17`.

Inputs: `research_measurements.csv`.

From a fresh copy of this folder, run:

```
python3 research_table.py
```

Rule to preserve. Each person-timepoint combination must be unique; allowed timepoints are baseline and followup. A supplied number must be finite. Change means follow-up minus baseline. Incomplete pairs remain in the person table but do not contribute to mean or median change.

17.2 Check the Result

Outputs: `derived_changes.csv`, `research_summary.csv`, `run_record.json`.

Check the meaning

Six people yield five complete pairs. Their changes are -2, -3, -3, -4, and -2: mean -2.8 and median -3.0. S04 remains with baseline 11, blank follow-up, and reason incomplete pair. Source row references identify the two contributing positions for each person.

17.3 Read the Key Lines

```

45 for key, values in sorted(people.items()):
46     baseline=values.get("baseline")
47     followup=values.get("followup")
48     complete=baseline is not None and followup is not None
49     derived.append(dict(person_ref=key,
50         baseline=" " if baseline is None else baseline,
51         followup=" " if followup is None else followup,
52         baseline_row=source_rows.get((key,"baseline"),""),
53         followup_row=source_rows.get((key,"followup"),""),
54         unit="teaching units",paired=int(complete),
55         change=followup-baseline if complete else "",
56         reason=" " if complete else "incomplete pair"))

```

Excerpt from research_table.py, source lines 45–56.

Use a compound identity. A nested dictionary stores values by person and timepoint. The duplicate check concerns that pair, not person alone; two records for the same person are expected when the timepoints differ.

Use None for an absent value. The program distinguishes `None` from a numerical 0. The test `baselineisnotNoneandfollowupisnotNone` accepts genuine zero measurements while excluding missing ones.

A derived record carries its sources. The table stores baseline and follow-up values, source row numbers, the paired flag, and the change. A summary can therefore be checked without guessing which rows were aligned.

17.4 Try One Change

Supply follow-up value 10 for S04's existing blank follow-up record. Expect six complete pairs, with S04's change -1. The mean and median both become -2.5. The original input and its run record must remain available as a separate version.

Reminder Simulation

The reminder simulation repeatedly generates two invented groups under supplied attendance probabilities. The saved trial table is the primary simulation record. Its summary, frequency table, and histogram are derived from those same trials rather than from separately generated samples.

The probabilities are assumptions. This program illustrates sampling variation under a model; it does not estimate the effect of a real reminder intervention.

18.1 Run the Example

Folder: `teaching_files/ch18`.

Inputs: `simulation_plan.csv`.

From a fresh copy of this folder, run:

```
python3 reminder_simulation.py
```

Rule to preserve. The input contains exactly one plan: positive group size and repetition count, probabilities between 0 and 1, and a seed. Each repetition independently generates the two groups using the declared model. Difference is reminder minus usual in percentage points. The two reported central bounds use the order-statistic indices written in the program.

18.2 Check the Result

Outputs: `simulation_frequency.csv`, `simulation_histogram.svg`, `simulation_summary.csv`, `simulation_trials.csv`.

Check the meaning

With 80 per group, probabilities 0.70 and 0.80, 1,000 repetitions, and seed 104, the reference run gives mean difference 10.08 points, central bounds -3.75 and 22.5, and 80 trials in which the reminder count is not higher. The frequency counts must sum to 1,000. The central range is a simulation summary, not an automatically justified confidence interval for a real effect.

18.3 Read the Key Lines

```
38 rng=random.Random(int(plan["seed"]))
39 trials=[]
40 for trial in range(1,repetitions+1):
41     usual=sum(rng.random()<p0 for _ in range(n))
42     reminder=sum(rng.random()<p1 for _ in range(n))
43     trials.append(dict(trial=trial,usual=usual,reminder=reminder,
44         difference_points=100*(reminder-usual)/n))
45 values=sorted(r["difference_points"] for r in trials)
46 low=values[int(.025*(len(values)-1))]
47 high=values[int(.975*(len(values)-1))]
```

Excerpt from reminder_simulation.py, source lines 38–47.

A local random generator has explicit state. `random.Random(seed)` creates the generator used by this simulation. Reusing the same implementation, seed, and sequence of draws supports repeatability. Adding an extra random draw can change later simulated results.

A comparison simulates one binary outcome. `rng.random()<p0` is one simulated outcome; summing such Boolean comparisons counts successes. The outer loop repeats the two-group experiment.

Quantile conventions are part of a specification. The code sorts differences and selects the entries at the displayed integer indices. Another percentile convention could give a different bound. The companion explains the implemented rule rather than claiming all percentile methods are interchangeable.

18.4 Try One Change

Set both supplied probabilities to 0 and keep the other plan fields unchanged. Every trial must have zero attendance in both groups, all differences and central bounds must be zero, and reminder-not-higher must count all 1,000 trials. This is a deterministic boundary test of a random program.

District Outreach

The district report contrasts crude percentages with a calculation using the same supplied age-band weights for every district. It makes clear that different population compositions can change crude comparisons even when a common-weight calculation is similar.

The two equal weights are classroom choices. The program does not claim that they are the appropriate standard population for a real public-health report.

19.1 Run the Example

Folder: `teaching_files/ch19`.

Inputs: `district_outreach.csv`.

From a fresh copy of this folder, run:

```
python3 outreach_rates.py
```

Rule to preserve. Each district must have exactly one under65 row and one 65plus row. Each eligible count is positive and reached lies between 0 and eligible. The crude percentage pools the district's counts. The standardized percentage is the equally weighted average of its two age-band percentages.

19.2 Check the Result

Outputs: `district_rates.csv`.

Check the meaning

North has crude 54.0% and standardized 70.0%; South has 78.0% and 70.0%; East has 72.0% and 70.0%; West has 56.0% and 60.0%. North's two supplied band rates are 50% and 90%, whose equal-weight average is 70%. The output keeps the weights visible.

19.3 Read the Key Lines

```
45     reached=sum(x[0] for x in bands.values())
46     eligible=sum(x[1] for x in bands.values())
47     rates={band:100*n/d for band,(n,d) in bands.items()}
48     report.append(dict(district=district,reached=reached,eligible=eligible,
49                        crude_percent=round(100*reached/eligible,1),
50                        standardized_percent=round(.5*rates["under65"]+.5*rates["65plus"],1),
51                        weight_under65=.5,weight_65plus=.5))
```

Excerpt from outreach_rates.py, source lines 45–51.

Two formulas answer different questions. The crude rate divides summed reached counts by summed eligible counts. The common-weight rate combines the separately calculated band rates. Replacing one formula with the other changes the question.

Keep assumptions beside results. The output includes both weights as explicit columns. A reader can therefore reconstruct the calculation instead of treating “standardized” as an unexplained label.

Rounding belongs at presentation. The program forms rates from unrounded counts and rounds the final reported percentages. The original counts remain available for a more precise recalculation.

19.4 Try One Change

Double both reached and eligible counts in both North rows. Expect North totals of 1,080 reached among 2,000 eligible, but unchanged crude 54.0% and standardized 70.0%. Scaling the counts should not alter their ratios. Do not apply the multiplier to only a numerator.

Journey Tests

The journey test runner checks the reusable programs against named expected cases. It tests more than whether the process completes: source conservation, exact review reasons, duplicate correction, date boundaries, visit anchors, input headers, record order, escaping, person selection, overwrite refusal, and repeated runs.

20.1 Run the Example

Folder: `teaching_files/ch20`.

Inputs: `journey_demo.csv`, `journey_problem.csv`.

Local modules: `journey_check.py`, `journey_report.py`, `journey_view.py`.

From a fresh copy of this folder, run:

```
python3 journey_test_runner.py
```

Rule to preserve. Each trial either completes its assertions or records a failure with a detail. The full result CSV is written even when one trial fails. Any failure produces a nonzero process exit code. Tests retain exact source values and identities rather than accepting an unchanged total alone.

20.2 Check the Result

Outputs: `journey_tests.csv`.

Check the meaning

The baseline has 12 PASS rows and no FAIL rows. In particular, **problem_reasons** verifies the six exact source rows and reasons; **person_selection** checks P03's mismatched record; **no_overwrite** compares the earlier files before and after a refused run. Read those tests, not just the final count.

20.3 Read the Key Lines

```
25 def require_retained(source, report):
26     require(len(report) == len(source), "All input records must remain")
27     require({r["data_row"] for r in report} == set(range(1, len(source)+1)),
28             "Source-row identities were lost or duplicated")
29     by_row = {r["data_row"]: r for r in report}
30     for number, original in enumerate(source, 1):
31         require(all(by_row[number][key] == value
32                    for key, value in original.items()),
33                "Source values changed in row " + str(number))
```

Excerpt from journey_test_runner.py, source lines 25–33.

Assert the identity, not only the size. `require_retained` checks row count, source positions, and every supplied field. A report with the right number of rows but altered values must fail.

Temporary folders isolate file tests. `TemporaryDirectory` supplies fresh destinations for tests of selection, overwriting, and repeatability. These tests do not need to erase a reader's previous output folder.

Test the tests with a deliberate defect. A suite that never fails may be checking too little. A controlled mutation of one rule, made only in a disposable copy, can show that at least the relevant check detects that defect. This is evidence about that mutation, not universal test coverage.

20.4 Try One Change

In a disposable copy of `journey_check.py`, change the event-before-visit comparison from `days[i]<days[anchor]` to `days[i]<=days[anchor]`. Run the suite in a fresh folder. It must report failure, including `same_day`; the defective comparison now rejects permitted same-day events. Restore the original program rather than changing expected answers to force a pass.

Demonstration Package

The package builder constructs a small demonstration ZIP from an explicit file list. It adds a manifest and instructions. Unrelated files sitting beside the program are not automatically swept into the handover.

21.1 Run the Example

Folder: `teaching_files/ch21`.

Inputs: `journey_demo.csv`, `journey_problem.csv`, `package_files.csv`, `expected_summary.txt`.

Files to include: `journey_check.py`, `journey_report.py`, `journey_view.py`.

From a fresh copy of this folder, run:

```
python3 package_builder.py
```

Rule to preserve. The package list must be nonempty, well formed, and free of duplicate or reserved filenames. Each listed name must be a simple filename from the same folder. Every listed file must exist. The output ZIP is created exclusively, and contains the six source files plus `manifest.json` and `RUN.txt`.

21.2 Check the Result

Outputs: `journey_demo_package.zip`.

Check the meaning

The ZIP has eight members. The source files are the three journey programs, the clean and problem CSVs, and `expected_summary.txt`. Extract it into a fresh folder, run the command in its `RUN.txt`, and check 18 records, 12 placed, and six for review. The manifest's file hashes identify the included bytes.

21.3 Read the Key Lines

```
18 manifest = []
19 for item in items:
20     name = item["filename"]
21     if not name.strip() or name != name.strip() or Path(name).name!=name:
22         raise ValueError("Use simple filenames from this folder")
23     data = (ROOT/name).read_bytes()
24     manifest.append(dict(filename=name,purpose=item["purpose"],
25                           bytes=len(data),sha256=hashlib.sha256(data).hexdigest()))
```

Excerpt from package_builder.py, source lines 18–25.

An explicit list is an inclusion rule. The program reads `package_files.csv`; it does not archive the entire directory. A new file placed in the working folder is therefore excluded unless deliberately added to the list.

Check names before reading files. The simple-filename check prevents the manifest from requesting an arbitrary relative path. Reserved names protect the generated manifest and instructions from collisions with supplied files.

A ZIP container has metadata as well as content. Two runs can produce different ZIP bytes because member timestamps differ. Compare member names and extracted member bytes when checking the packaged content; do not label every container-byte difference a source change.

21.4 Try One Change

Create an unrelated file named `decoy.txt` beside the program, but do not add it to `package_files.csv`. Build the ZIP in a fresh copy. It should still have eight members and no `decoy.txt`. This checks that the explicit inclusion rule controls the package.

Review-Status Check

The review-status checker organizes an invented register of review records. It asks whether the listed version, decision, reviewer role, and validity date meet the supplied completeness rule. It cannot decide whether a review was adequate or grant approval for real use.

22.1 Run the Example

Folder: `teaching_files/ch22`.

Inputs: `review_register.csv`.

From a fresh copy of this folder, run:

```
python3 review_status.py
```

Rule to preserve. The register must contain exactly one row for each of Specification, Program, Tests, Data handling, and User guide. A complete record needs an accepted decision, matching nonblank versions, a reviewer role, and a valid-until date not earlier than September 17, 2026. Several blocker reasons can belong to one artifact.

22.2 Check the Result

Outputs: `review_status_report.csv`.

Check the meaning

Specification and User guide are complete. Program has a version mismatch; Tests has both an unaccepted review and a missing/mismatched version; Data handling has an expired review. Thus three records have blockers but there are four reasons. Every output row must retain `as_of=2026-09-17` and `rule_version=review-register-v1`.

22.3 Read the Key Lines

```

43 for row in rows:
44     issues=[]
45     if row["decision"]!="accepted":
46         issues.append("review not accepted")
47     if (not row["current_version"].strip() or not row["reviewed_version"].strip()
48         ↪ )
49         or row["current_version"]!=row["reviewed_version"]):
50         issues.append("missing or mismatched version")
51     if not row["reviewer_role"].strip():
52         issues.append("reviewer missing")
53     try:
54         if parse_day(row["valid_until"])<AS_OF:
55             issues.append("review expired")
56     except ValueError:
57         issues.append("invalid review date")

```

Excerpt from review_status.py, source lines 43–56.

A set comparison checks the inventory. The combination of a duplicate check and equality with the required artifact set ensures neither omissions nor extra categories. A count of five alone would not establish the correct inventory.

Accumulate independent blockers. Separate conditions append independent issues. Replacing them with an `elif` chain would lose later reasons after the first problem was found.

Use a fixed reporting date for a reproducible example. `AS_OF` is supplied in the program, not taken from the computer clock. Repeating the example tomorrow therefore uses the same contract unless that value is deliberately changed.

22.4 Try One Change

In the invented Program row, change `reviewed_version` from 3 to 4 so it matches `current_version`. Expect three complete records and two blocker records. Tests still has its two reasons; the expired Data handling review is unchanged. This input change does not itself establish that a real review occurred.

Follow-Up Worklist

The final chapter classifies an invented administrative encounter file and writes both a CSV worklist and a browser view. Status and owner controls filter the displayed rows; they do not alter the underlying records. The program sends no messages, changes no appointments, and assigns no clinical priority.

23.1 Run the Example

Folder: `teaching_files/ch23`.

Inputs: `worklist_encounters.csv`.

From a fresh copy of this folder, run:

```
python3 followup_worklist.py
```

Rule to preserve. Encounter IDs are distinct. The default reporting date is September 17, 2026, and the window is 14 calendar days. Invalid dates, missing contact/owner details, future index dates, or inconsistent follow-up dates cause review data. Otherwise a supplied valid follow-up is recorded, even if late; without one, the window is passed only when the reporting date is strictly later than its end.

23.2 Check the Result

Outputs: `followup_worklist.csv`, `followup_worklist.html`, `worklist_run.json`.

Check the meaning

E01 and E06 have follow-up recorded. E02 has no follow-up after its window. E03 remains open until September 24 inclusive. E04 has a follow-up before its index date and requires review; E05 requires review for a missing contact route. All six encounter IDs must remain. A recorded late follow-up and a within-window match are different concepts, as the capstone demonstrates.

23.3 Read the Key Lines

```
63     if issues:
64         status = "review data"
65     elif followup:
66         status = "follow-up recorded"
67     elif as_of > due:
68         status = "no follow-up after window"
69     else:
70         status = "window still open"
71     report.append(dict(row, data_row=number, as_of=as_of.isoformat(),
72                       window_days=window, rule_version=VERSION,
73                       due_date=due.isoformat() if due else "", status=status,
74                       reason="; ".join(dict.fromkeys(issues))))
```

Excerpt from followup_worklist.py, source lines 63–74.

Decision precedence prevents misleading labels. The `if issues` branch comes before the recorded, passed-window, and open-window branches. A date or contact problem must not be hidden behind an otherwise plausible administrative label.

A strict comparison keeps the end date inclusive. `as_of>due` means the window is still open on its end date. Changing this to greater-than-or-equal would alter the boundary contract.

Filtering is not deletion. JavaScript hides rows that do not match the selected status and owner. The count and empty-state message describe the visible subset. The CSV and run record remain the complete report.

23.4 Try One Change

Run `python3followup_worklist.py--as-of2026-09-25` in a fresh copy. E03 should now have no follow-up after window, giving two recorded, two passed-window, zero open, and two review records. On September 24 itself, E03 is still open under the inclusive-end rule.

The Integrated Capstone

The capstone connects the matcher, weekly monitor, and worklist through an explicit field map. It uses a new invented dataset, not an assumed connection between unrelated chapter files. The three application programs are unchanged copies from Chapters 8, 11, and 23. An adapter prepares their inputs; a test program checks the integrated results.

Run the Fixed Example

Copy `teaching_files/capstone` in full. Its inputs are `capstone_encounters.csv`, `capstone_followups.csv`, and `capstone_weeks.csv`. Keep all five Python files and the text instructions together. From the fresh copy, run these commands in order:

```
python3 capstone_adapter.py
python3 capstone_tests.py
```

The adapter creates a new `integrated_run` folder. The tests read it and write `capstone_tests.csv`. Alternatively, from the companion root, use:

```
python3 run_companion.py capstone --output capstone_trial
```

Keep the Three Meanings Separate

The fixed Stage-A reporting date is September 17, 2026. Matching includes elapsed days 0 through 14. The weekly denominator contains all supplied encounters in that dated week. Twenty encounters and twenty follow-up records make a 40-row source ledger; two weekly status flags are separate inputs.

A within-window match is not the same as a recorded follow-up. C103 has valid follow-up at day 16: recorded, but not matched within the window. C114 has a within-window match but lacks a contact route: it is also a review case. A final weekly flag does not imply that every encounter's window has closed.

Read the Field Map

Keep the source rows. The adapter records the original filename, row number, ID, and values. Both occurrences of F09 remain identifiable. Unknown C999 is not converted to a known encounter. Ambiguous or invalid follow-ups remain in the ledger even when they cannot be passed to the matcher.

Prepare each application's input. The matcher receives encounter IDs, person references, index dates, and usable follow-up candidates. Matching is by encounter ID, never by person alone. The worklist receives the earliest valid recorded follow-up on or after the index date and on or before the reporting date, without imposing the matching window. Upstream problems are passed separately to **classify**.

Make weekly counts traceable. The adapter groups encounters by the Monday of their index date. Its `week_map.csv` links dates to the earlier monitor's week numbers. Within-window selections supply the numerator; the denominator ledger identifies the encounters counted. `integration_record.json` records the date, window, denominator choice, interpreter, hashes, and counts.

The adapter rejects an invalid encounter index date because it cannot assign that encounter to a week. This is stricter than the standalone worklist's row-review rule. Reusing a module does not make every input contract identical.

Check the Result

The baseline retains 40 source records and all 20 encounters. It has 11 within-window matches. The worklist has 11 recorded follow-ups, one passed window, one open window, and seven review cases. The two sets of eleven are not identical: the recorded-follow-up set replaces C114 with C103.

The week beginning August 31 has 8 matches among 16 encounters, including 6 review cases. The provisional week beginning September 7 has 3 among 4, including 1 review case. Only the final week contributes to the pooled 50.0%. The tests should report 12 PASS and no FAIL results.

Check Identities, Not Just Totals

Check the meaning

Matched encounters: C101, C102, C107, C111, C112, C113, C114, C115, C116, C117, C118.

Review encounters: C104, C105, C106, C108, C109, C114, C120.

C110 alone has a passed window; C119 alone has a window still open.

The 12 tests cover the fixed contract, retained source and encounter identities, matching and status decisions, late follow-up, overlapping review and match flags, boundaries, duplicate and unknown references, dates, weekly counts, and dated outputs. Read the named assertions in `capstone_tests.py` when a check fails.

Saved comparison files are under `reference_outputs/capstone`. An actual run also creates staged inputs and execution transcripts; these working files are not all repeated in the curated reference folder.

Try a Different Contract

For Stage B, record the new decisions in a copy of `PROJECT_CARD.txt`. For example, run this in a fresh capstone copy:

```
python3 capstone_adapter.py --output changed_run \  
    --window-days 21 --denominator closed
```

The backslash continues the command onto the next line. Here `closed` means that the reporting date is strictly later than the window end. Under that denominator choice, the weekly numerator and review count also use only included encounters. The full source ledger and worklist remain complete.

Do not apply the fixed Stage-A expectations to a changed contract and edit `FAIL` into `PASS`. Predict new named cases, justify their expected results, and keep the original tests. The complete adapter, tests, field map, and specification remain in the capstone folder; they need not be printed here.

Larger Practice Files

Four independent fixtures extend the small chapter examples. Their programs are unchanged; their inputs and expected results are different. Each folder has its own `EXPECTED.json`. Never compare these runs with the smaller chapter's totals.

Run the commands below from the extracted companion folder, using new output names. Each helper command copies and runs the complete practice folder.

Chapter 2: More Journey Records

```
python3 run_companion.py practice02 --output journey_practice
```

The 36 source records yield 30 placed events and six review rows. The exact review source rows are 5, 8, 20, 27, 29, and 36. Check both the total and those positions. A familiar-looking ID does not link this independent fixture to Chapter 1's events.

Chapter 8: Competing Matches

```
python3 run_companion.py practice08 --output matching_practice
```

There are 16 encounters and 24 follow-up records. Fifteen encounters match; C012 does not. F24 has an unknown encounter reference. Compare the complete selection map in `practice_datasets/ch08/EXPECTED.json` with the actual selected IDs, not just the matched count.

Chapter 9: Different Measurement Populations

```
python3 run_companion.py practice09 --output cohort_practice
```

Forty candidates yield 32 eligible people, 24 recorded follow-ups, and 24 observed waits. The median observed wait is 10 days. The two equal counts of 24 do not establish that the same people supplied both. Inspect the source and disposition ledger.

Chapter 23: Overlapping Review Reasons

```
python3 run_companion.py practice23 --output worklist_practice
```

All 36 encounters remain, with nine in each of the four statuses. The recorded group is W001, W005, W009, W013, W017, W021, W025, W029, W033. The passed-window group begins W002 and continues every fourth ID through W034; the open-window group begins W003 and continues through W035; the review group begins W004 and continues through W036.

Use the explicit lists in `practice_datasets/ch23/EXPECTED.json` to check every status. A correct distribution of nine records per group could still conceal incorrect record assignments.

Make a New Test Deliberately

Keep the supplied fixture and its expected-result file unchanged. In a new copy, choose one record and one field, predict which named outputs should change, and then run. Check records that should remain unchanged as well. The existing expected-result file is a reference for the original fixture, not an answer key for every modified dataset.

A Small Python Reference

This optional reference explains the notation used in the excerpts. It is not a separate Python course. Return to the named application when you need a concrete example.

Values and Collections

A name such as `rows` refers to a value. Assignment uses `=`; equality comparison uses `==`. CSV fields are text: `"14"` is not the number 14 until an explicit conversion such as `int` or `float` succeeds. Blank text, zero, and `None` have different meanings.

A dictionary connects names to values: `row["encounter_id"]` retrieves one named field. A list retains a sequence of rows; a set retains distinct values. A tuple such as `(measure, unit)` can be one grouping key. `defaultdict(list)` collects groups; `Counter` counts occurrences. See Chapters 1, 2, and 5. Counting distinct people is not counting source rows.

Conditions, Loops, and Selection

An `if/elif/else` chain chooses one branch, as in the cohort or worklist. Independent `if` statements can collect several reasons, as in the review-status checker. These are different decisions.

A `for` loop visits a sequence. `enumerate(rows, 1)` supplies a row number as well as the row. A comprehension constructs a list, set, or dictionary from selected values. Follow the inclusion condition before admiring the compact notation. Chapter 9 first selects eligible people, then their observed waits; those are not the same population.

A sorting key determines order. Chapter 8 sorts `(days, followup_id)` pairs, so the ID resolves an elapsed-day tie. Changing that key can change the selected follow-up even when no input row is removed.

Functions and Modules

A function groups instructions for a named job and may return a result. Arguments provide its inputs. A default such as `person=None` applies when the caller supplies no value for that argument.

An `import` makes a module available and executes its top-level statements. Reusable modules put their entry point behind `if __name__ == "__main__":`. Several short chapter scripts instead do their work at top level. Keep required local modules together, but do not import every chapter script merely to inspect it.

Files and Failed Runs

`csv.DictReader` reads named fields; `csv.DictWriter` writes them. The supplied programs also check exact headers and malformed rows. Their file paths generally locate inputs beside the script. Chapter 3 additionally accepts command-line paths, interpreted from the working directory when relative. A correct filename in the wrong folder is still missing.

Exclusive creation mode "`x`" refuses an existing file. A failed run can leave partial output. Preserve it, read the error, and use a new destination; do not assume that every multi-file operation either succeeds entirely or leaves nothing behind.

A `try/except` block handles a specified exception. A value problem can become a row-level reason only after the program knows which fields the row contains. The last line of a traceback is a useful starting point, not a reason to change the stated data rule.

Dates and Numerical Summaries

Date arithmetic supplies elapsed days; the comparison supplies the rule. `0<=days<=14` includes both endpoints. `as_of>due` keeps the window open through its end date. See Chapters 8 and 23.

Mean and median operate on the values supplied to them. State which records contribute. An empty median is not zero, and an absent denominator does not give a 0% rate. Pool counts before dividing when the question concerns a pooled proportion; do not average rounded weekly percentages.

Tests and Reproduction

A test compares an actual result with an explicit expectation. Check source identities and values as well as counts. The Chapter 20 deliberate defect shows why testing the tests can be useful.

A hash identifies file bytes, not their correctness. For a simulation, record the seed, interpreter, inputs, and program. For a ZIP, distinguish container metadata from member contents. For a browser page, distinguish the saved data from the currently displayed subset. These are different things to compare.

Checks and Source Files

What Changed

This edition removes the complete printed listings, raw input tables, repeated execution summaries, and separate part-title pages. It retains all 23 applications, their required files and commands, named-result checks, selected source excerpts, one change per application, the capstone, and the four larger practice fixtures. The complete runnable files remain in the ZIP.

What Was Checked Again

All chapter, capstone, and practice programs were run in fresh copies. The journey and capstone test suites each reported 12 passes and no failures. File comparisons checked that the programs, input data, and supplied reference outputs were unchanged from the uploaded companion. Each printed excerpt was compared with its stated source lines. The revised LaTeX was compiled and the PDF checked for layout.

CHECKED_RESULTS.txt records the scope of these checks and preserves the original release's more detailed checking record separately. Historical browser checks in that record are not new cross-browser or usability tests. A successful execution does not establish clinical validity, privacy compliance, or suitability for real operations.

Edit or Typeset

Keep `Python_Companion.tex` and `Python_Companion.sty` in one folder. Run pdfLaTeX twice, or use latexmk, to resolve the contents and references. The manuscript embeds its short excerpts; the `.py` and `.csv` files are for execution and editing, not typesetting inputs. Edit the full program file when changing an application, not just its printed excerpt.

Further Reference

The application rules and programs come from the supplied medical-book release named at the front. The following Python 3.13 documentation links, retained from the source edition, explain the standard-library tools rather than validating the applications:

Python Software Foundation, *CSV File Reading and Writing*.

<https://docs.python.org/3.13/library/csv.html>

Python Software Foundation, *Basic Date and Time Types*.

<https://docs.python.org/3.13/library/datetime.html>

Python Software Foundation, *Mathematical Statistics Functions*.

<https://docs.python.org/3.13/library/statistics.html>